DIGITAL SIGNAL PROCESSING

ASSIGNMENT REPORT

Submitted By

Mohamed Shanil

B23ECA45

Semester 5

Batch A

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

T K M COLLEGE OF ENGINEERING

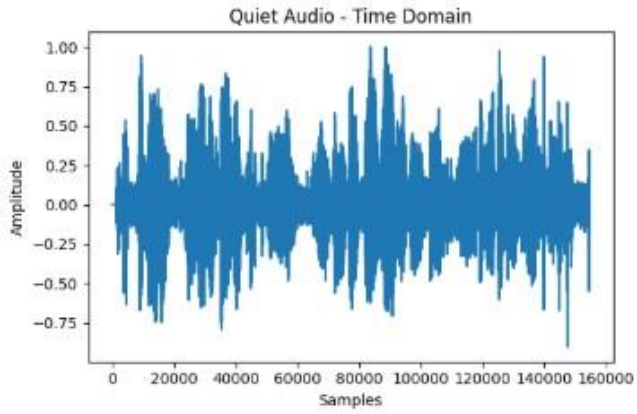
KOLLAM 691005

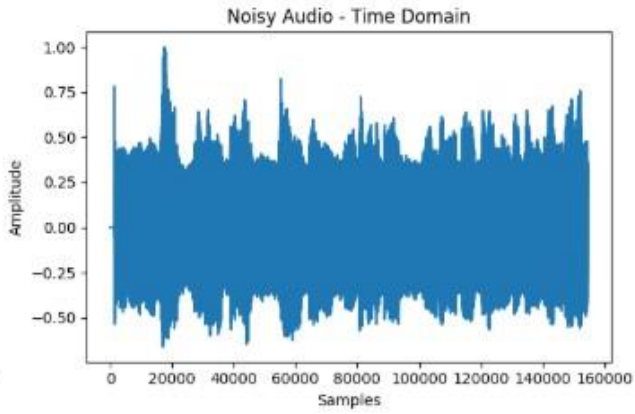
ROLL NO: B23ECA45 , T5A

INTRODUCTION

This project focuses on analyzing and filtering real-world audio using the Discrete Fourier Transform (DFT), specifically the Fast Fourier Transform (FFT). We recorded audio in both quiet and noisy environments to study how noise affects frequency content. Using frequencydomain filtering, we removed unwanted components and reconstructed a cleaner signal. This assignment demonstrates how signal processing can enhance audio quality in practical applications like hearing aids and smart acoustic systems.

```
import librosa import soundfile
as sf import matplotlib.pyplot
as plt from IPython.display
import Audio

# Load the quiet and noisy audio quiet_audio, sr =
librosa.load("quiet.wav", sr=16000) noisy_audio, sr
= librosa.load("noisy.wav", sr=16000)

# Normalize quiet_audio = quiet_audio /
np.max(np.abs(quiet_audio)) noisy_audio = noisy_audio
/ np.max(np.abs(noisy_audio))
# Make both audio signals same length for FFT
comparison min_len = min(len(quiet_audio),
len(noisy_audio)) quiet_audio = quiet_audio[:min_len]
noisy_audio = noisy_audio[:min_len]

# -------------------------
# Step 2: Plot Time Domain Signals #
-------------------------
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(quiet_audio)
plt.title("Quiet Audio - Time
Domain") plt.xlabel("Samples")
plt.ylabel("Amplitude")
 plt.subplot(1, 2, 2)
plt.plot(noisy_audio)
plt.title("Noisy Audio - Time
Domain") plt.xlabel("Samples")
plt.ylabel("Amplitude")
plt.tight_layout() plt.show()
```



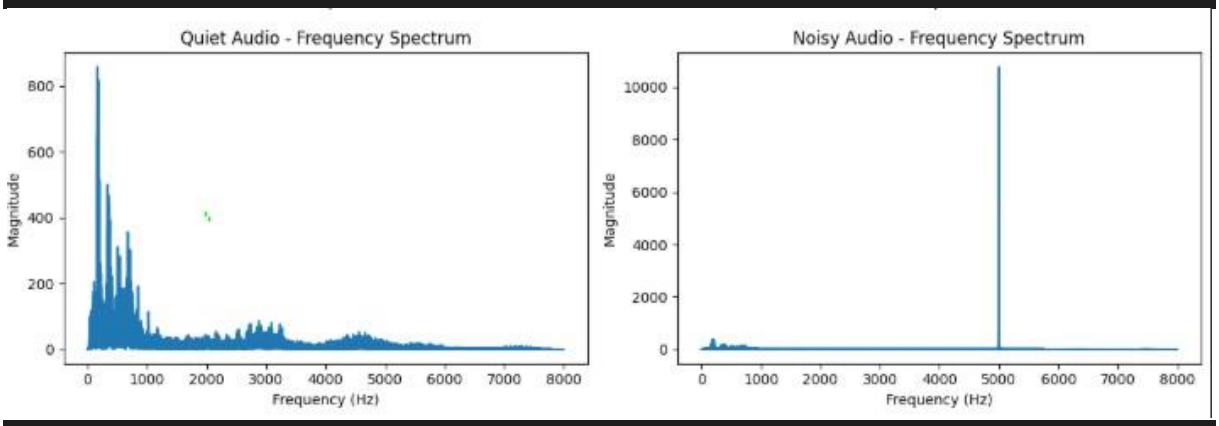


FFT STEPS

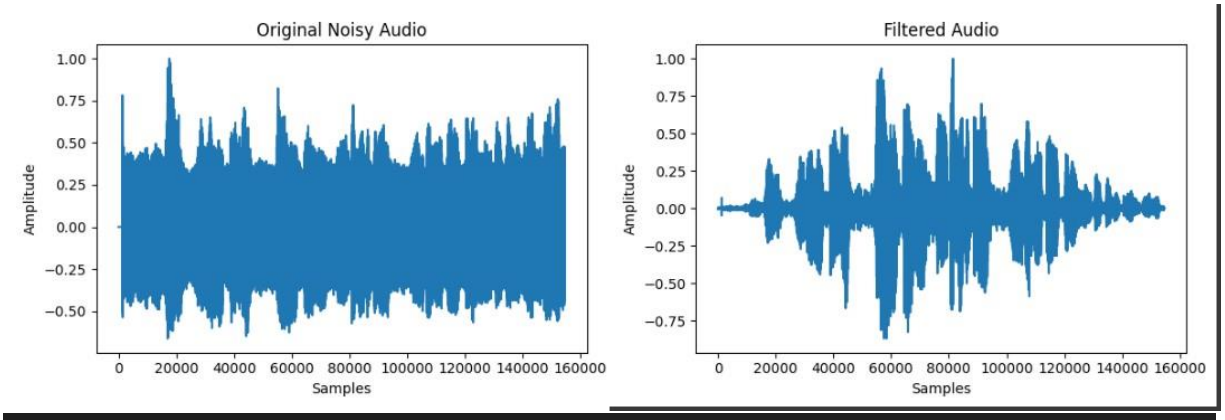
To analyze the spectral content of the audio signals, we applied the Fast Fourier Transform (FFT), which converts time-domain signals into the frequency domain. The FFT output is a complex-valued spectrum, from which we extracted the magnitude to study how energy is distributed across various frequencies. In the quiet audio, the frequency spectrum displayed a structured pattern with distinct dominant regions. In contrast, the noisy audio exhibited broad and irregular high-magnitude components, indicating the presence of unwanted noise spread across a wide frequency ranges.

```
# Step 3: FFT and Frequency Spectrum
# ------------------------- # Apply Hamming window
quiet_windowed = quiet_audio *
np.hamming(len(quiet_audio)) noisy_windowed = noisy_audio
* np.hamming(len(noisy_audio))

# FFT
X_quiet = np.fft.fft(quiet_windowed)
X_noisy = np.fft.fft(noisy_windowed) freqs
= np.fft.fftfreq(len(X_noisy), 1/sr)

# Plot magnitude spectrum plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1) plt.plot(freqs[:len(freqs)//2],
np.abs(X_quiet[:len(freqs)//2])) plt.title("Quiet Audio -
Frequency Spectrum") plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
 plt.subplot(1, 2, 2) plt.plot(freqs[:len(freqs)//2],
np.abs(X_noisy[:len(freqs)//2]))
```

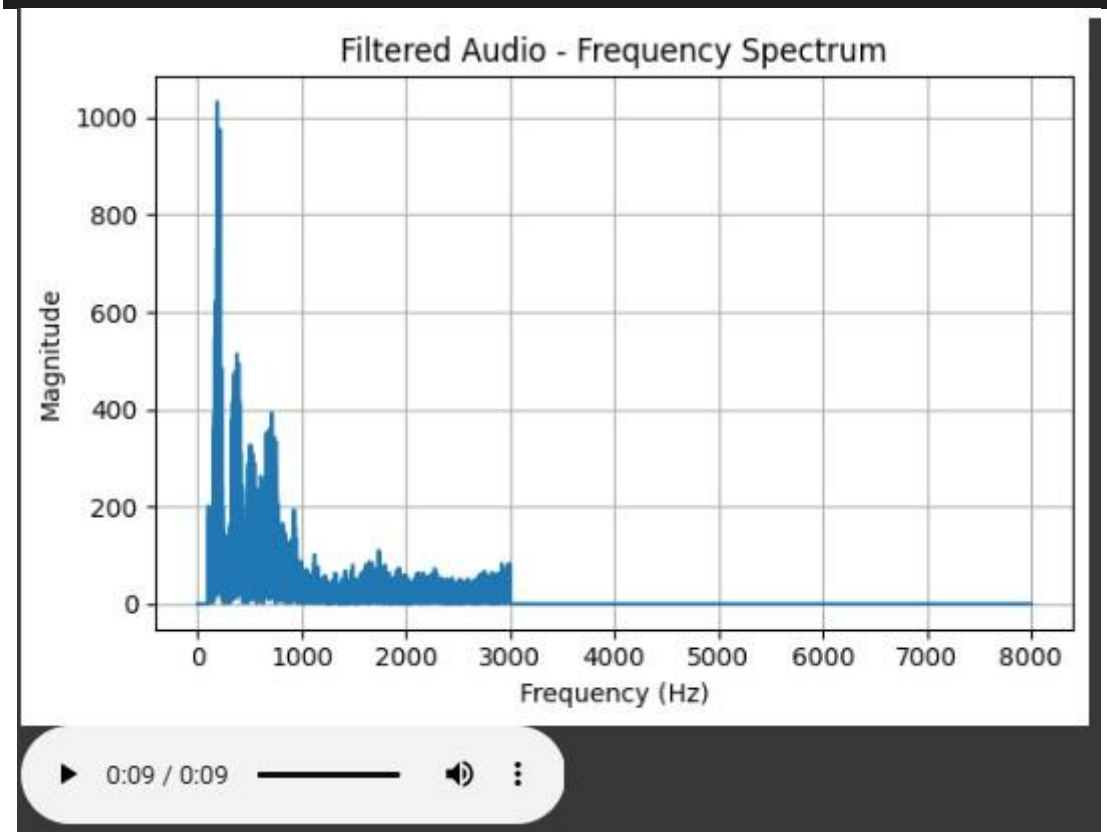
```
plt.title("Noisy Audio - Frequency Spectrum")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude") plt.tight_layout()
plt.show()

# -------------------------
# Step 4: Frequency-Domain Filtering
# -------------------------
# Zero out frequencies below 100Hz and above 3000Hz
X_filtered = X_noisy.copy()
X_filtered[(freqs < 100) | (freqs > 3000)] = 0

#     Inverse     FFT     filtered_audio     =
np.fft.ifft(X_filtered)   filtered_audio   =
np.real(filtered_audio)
filtered_audio = filtered_audio / np.max(np.abs(filtered_audio))  #
Normalize

#  Save  filtered  audio  sf.write("filtered_noisy.wav",
filtered_audio, sr)
```




```
# Plot comparison
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(noisy_audio)
plt.title("Original Noisy Audio")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
 plt.subplot(1, 2, 2)
plt.plot(filtered_audio)
plt.title("Filtered Audio")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
plt.tight_layout()

plt.show()
```




```
 Step 5: Frequency Spectrum of Filtered Audio
# -------------------------
X_filtered_mag = np.fft.fft(filtered_audio) freqs_filtered
= np.fft.fftfreq(len(X_filtered_mag), 1/sr)
 plt.figure(figsize=(6, 4))
plt.plot(freqs_filtered[:len(freqs_filtered)//2],
np.abs(X_filtered_mag[:len(freqs_filtered)//2]))
plt.title("Filtered Audio - Frequency Spectrum")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude") plt.grid()
plt.tight_layout() plt.show()
```



Filtering Method Explanation

To reduce noise in the audio signal, we applied a frequency-domain filtering approach using FFT. Specifically, we eliminated all frequency components below 100 Hz and above 3000 Hz by setting those frequency bins to zero. This allowed us to retain only the most relevant frequency range for human speech, which typically lies between 100 Hz and 3 kHz. Once the unwanted frequencies were removed, we used the inverse FFT (IFFT) to convert the signal back to the time domain. Finally, the reconstructed signal was normalized again to ensure consistent amplitude before playback.

Comparison of Spectrum

In the quiet environment, the magnitude spectrum revealed that most of the signal's energy was concentrated between 150 Hz and 3 kHz—an expected range for human speech. Higher frequency regions contained very little energy, indicating a clean acoustic setting. In contrast, the spectrum of the noisy signal showed prominent low-frequency components, likely caused by background sources such as fans or electrical hums. Additionally, there were noticeable high-frequency spikes above 3 kHz, pointing to the presence of disruptive noise. This contrast highlights how FFT analysis can uncover hidden spectral details, making it easier to differentiate between meaningful speech signals and unwanted noise.

AUDIO LINKs

Quiet voice

https://drive.google.com/file/d/1hTGndWerqD0m8BDkuJnIvYl6IenYKbq7/view?usp=drivesdk

Noisy Voice

https://drive.google.com/file/d/1hXJrVF4M6fUuJAmAT9QWH3v_4eKzdqG/view?usp=drivesdk

Filtered voice

https://drive.google.com/file/d/1hYqkF6-MNZRQkYZXFVRoqpD6xIACs_Ha/view?usp=drivesdk

Summary of Methods and Findings

We recorded two audio clips: one in a quiet environment and one in a noisy setting. These signals were normalized and analyzed in both time and frequency domains using FFT. We observed that the noisy signal exhibited significant high- and low-frequency components. Using a frequency-domain zeroing technique, frequencies below 100 Hz and above 3000 Hz were removed. The inverse FFT was applied to reconstruct the time-domain signal. The filtered audio showed improved clarity by reducing unwanted noise components.

SDG 3: Good Health and Well-being

Focus: Assistive Hearing Technologies

Real-world signal analysis directly supports SDG 3 by enabling the creation of assistive hearing technologies that enhance speech perception and reduce background noise. Individuals with hearing impairments often face challenges in understanding speech in noisy environments such as classrooms, public places, or transportation hubs. Through digital signal processing techniques like FFT and noise filtering, hearing aids and assistive devices can isolate and amplify relevant speech frequencies while suppressing noise. This makes conversations clearer and less mentally tiring for the listener. Moreover, adaptive filtering and real-time noise cancellation allow for more personalized and comfortable listening experiences. These advancements not only improve auditory accessibility but also contribute to better mental health, social participation, and quality of life. By making hearing assistance more effective and accessible, signal processing plays a key role in inclusive healthcare technologies, aligning directly with the goals of SDG 3: ensuring healthy lives and promoting well-being for all at all ages.

SDG 9: Industry, Innovation and Infrastructure

Focus: Acoustic Sensing for Smart Infrastructure

SDG 9 focuses on building resilient infrastructure and fostering innovation, both of which are supported by real-world signal analysis. In smart infrastructure systems, acoustic sensing and signal processing are used to monitor environments in real time. For instance, audio sensors can detect mechanical faults in bridges, tunnels, or machines by analyzing vibration and sound patterns. In smart buildings, sound-based monitoring can detect unusual activities, optimize energy use, or improve public safety. Signal processing techniques like Fourier analysis, filtering, and pattern recognition help interpret these audio signals accurately and quickly. This data-driven approach enables predictive maintenance, reduces downtime, and minimizes the risk of catastrophic failures. Additionally, acoustic analysis supports urban planning by monitoring traffic noise, crowd density, and environmental soundscapes. By embedding such intelligent sensing systems in infrastructure, we make cities more efficient, safe, and innovative—thus fulfilling SDG 9's aim of building sustainable industry and infrastructure powered by cutting-edge technology.

SDG 12: Responsible Consumption and Production

Focus: Efficient Audio Processing for Sustainability

SDG 12 emphasizes the importance of reducing resource use and environmental impact. Signal processing directly contributes to this by enabling more **efficient audio processing**, especially in consumer electronics like smartphones, smart speakers, and wearable devices. Techniques such as spectral analysis, compression, and frequency-domain filtering allow devices to handle audio more intelligently while consuming less power. For example, using real-time FFT-based filtering reduces the need for high computational overhead, thereby extending battery life and reducing energy demand. This contributes to lower heat output and longer hardware lifespan, minimizing electronic waste. In large-scale applications like audio streaming services, optimized signal processing algorithms reduce bandwidth usage and server load. Overall, efficient signal analysis ensures that digital systems can perform complex audio tasks with minimal environmental cost. By embedding sustainability into everyday technology through smart engineering, signal processing plays a vital role in supporting SDG 12's vision of responsible and sustainable production and consumption.

Conclusion

This project demonstrated how FFT-based frequency filtering can effectively reduce noise from audio signals. By removing frequencies outside the 100 Hz to 3000 Hz range, we enhanced the clarity of speech in noisy recordings. The results highlight the practical value of digital signal processing in real-world applications like assistive hearing devices and smart acoustic systems